

Домашняя работа по Методам Списков и Срезам (Средний и Высокий уровни)

● Средний уровень (Комбинирование методов)

Задача 1: Анализ журнала регистрации

Вы анализируете журнал регистрации действий пользователей, где записи содержат дубликаты и ошибки.

Входные данные:

```
log_entries = ['login', 'logout', 'error', 'login',  
'timeout', 'logout', 'error', 'login']
```

Требуется:

1. Подсчет ошибок: Используя метод `count()`, посчитайте общее количество записей `'error'`.
2. Поиск и удаление: Найдите индекс первого вхождения `'timeout'` и, используя `pop()`, удалите эту запись из журнала.
3. Оценка эффективности: Выведите, сколько раз произошло действие `'login'`, а также длину списка после удаления.

Ключевые методы: `count()`, `index()`, `pop()`.

Задача 2: Обновление плейлиста

Вы управляете плейлистом и получили список новых песен, но несколько старых треков в середине плейлиста нужно заменить.

Входные данные:

```
playlist = ['Song A', 'Song B', 'Song C', 'Song D', 'Song  
E', 'Song F']  
new_tracks = ['New 1', 'New 2', 'New 3']
```

Требуется:

1. Замена среза: Используя срезы, замените треки с индекса 2 по 4 (`'Song C'`, `'Song D'`, `'Song E'`) на содержимое списка `new_tracks`.
2. Добавление в конец: Используя `append()` и `extend()`, добавьте в конец плейлиста трек `'Bonus Track'` (через `append()`), а затем список `['Final A', 'Final B']` (через `extend()`).
3. Выведите финальный плейлист.

Ключевые методы: Срезы для замены `[] = ...`, `append()`, `extend()`.

Задача 3: Резервное копирование и сортировка результатов

Вам пришли результаты тестов, и их нужно сначала отсортировать, но перед этим обязательно сохранить их оригинальный, несортированный порядок для резервного копирования.

Входные данные:

```
results = [88, 95, 76, 95, 65, 80]
```

Требуется:

1. Копирование: Создайте копию списка `results` под названием `backup_results` с помощью метода `copy()`.
2. Сортировка: Отсортируйте исходный список `results` по убыванию (от большего к меньшему).
3. Переворот: Используйте метод `reverse()` на списке `backup_results`, чтобы увидеть исходные результаты в обратном порядке ввода.
4. Выведите оба списка, чтобы подтвердить, что они разные.

Ключевые методы: `copy()`, `sort(reverse=True)`, `reverse()`.

Высокий уровень (Множественные операции, Сложные срезы)

Задача 4: Фильтр и очистка данных

Вы получили список идентификаторов, но в нем есть недействительные значения ('INVALID') и лишние пробельные символы.

Входные данные:

```
raw_ids = [' ID-101 ', 'ID-203', 'INVALID', ' ID-305 ',  
'ID-404', 'INVALID']
```

Требуется:

1. Очистка: Создайте цикл или используйте генератор списков (если проходили), чтобы пройти по `raw_ids` и очистить каждый элемент от начальных и конечных пробелов с помощью метода `strip()`. Сохраните в `clean_ids`.
2. Удаление по значению: Используйте цикл `for` или операцию `while` и метод `remove()`, чтобы удалить все вхождения 'INVALID' из списка `clean_ids`. (Внимание: использование `remove()` в цикле требует осторожности или знания, как работать с `while`.)
3. Выведите финальный список `clean_ids` и количество удаленных элементов.

Ключевые методы: strip() (применение в цикле), remove(), count().

Задача 5: Ротация и Шифрование

Для простого шифрования нужно взять список символов, перевернуть его, а затем поменять местами первую и последнюю половины списка.

Входные данные:

```
cipher_list = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
```

Требуется:

1. Переворот: Создайте перевернутую копию списка `cipher_list` с помощью среза `::-1`. Сохраните в `reversed_cipher`.
2. Разделение: Разделите `reversed_cipher` на две равные части (`first_half` и `second_half`). Используйте `len()` и срезы.
3. Ротация: Объедините обе половины, но поменяйте их местами (`second_half` идет первой), используя оператор `+`. Сохраните в `rotated_cipher`.

Ключевые методы: Срезы `::-1`, `len()`, конкатенация `+`, срезы для разделения.

Задача 6: Проверка контрольной суммы и вставка

Вы проверяете данные. Контрольная сумма должна быть равна сумме первого и последнего элементов. Если сумма не совпадает, ее нужно вставить в список на третью позицию.

Входные данные:

```
data_check = [10, 5, 20, 30]
```

(Здесь 30 — контрольная сумма)

Требуется:

1. Извлечение: Извлеките первый элемент (`data_check[0]`) и последний элемент (`data_check[-1]`).
2. Проверка: Вычислите ожидаемую сумму (первый + второй элемент, т.е. $10+5=15$). Сравните ее с контрольной суммой (последним элементом, т.е. 30).
3. Действие: Если ожидаемая сумма не равна контрольной сумме:
 - Удалите неверную контрольную сумму (последний элемент) с помощью `pop()`.
 - Вставьте правильную сумму (ожидаемую сумму) на индекс 2 (третья позиция).

4. Выведите финальный список и результат проверки.
Ключевые методы: Индексация, pop(), insert().